UAV Trajectory Optimization for Large-Scale and Low-Power Data Collection: An Attention-Reinforced Learning Scheme

Yuchen Zhu, Bo Yang, *Member, IEEE*, Min Liu, *Senior Member, IEEE*, and Zhongcheng Li

***Abstract*— Unmanned Aerial Vehicles (UAVs) exhibit great advantages in data collection from ground sensors in vast tracts of fields. Due to their limited power supply, most works assume that the UAV simply traverses each sensor's fixed transmission range to collect data, thereby shortening the flight path. However, they neglect the quality of collected data, which may deteriorate dramatically as the transmission distance increases. In this paper, by leveraging the physical-layer protocol - LoRa, we propose a Packet Reception Ratio (PRR)-based probabilistic coverage model to evaluate the quality of data transmission, which directly determines the data acquisition efficiency. On this basis, to minimize the energy consumption of UAV and sensors while ensuring high-quality data acquisition, we formulate the UAV trajectory planning as a joint Energy Consumption and data Acquisition Efficiency (ECAE) optimization problem. To tackle the ECAE problem, we propose a Deep Reinforcement Learning (DRL)-based two-stage scheme. First, an attention-based encoder-decoder model is trained to generate an initial trajectory. Then an intuitive optimization algorithm is devised to further explore the optimal trajectory. Evaluation results show that our scheme can reduce the total energy cost of UAV and sensors by 27.1% as compared to the best baseline's policy while maintaining a promising PRR.**

***Index Terms*— Unmanned aerial vehicle, trajectory optimization, data collection, attention model, deep reinforcement learning, LoRa.**

I. Introduction

INTERNET of Things (IoTs) technologies are becoming more attractive and widely applied in various vertical markets including industry, transportation, healthcare and agriculture. In an agricultural environment, typically with a wide geographical area, diverse sensors are deployed to monitor and collect the environment information (e.g., light, humidity, temperature, and soil moisture) [1]. The collected information needs to be delivered to the remote data center for decision support so as to manage the smart farms (e.g., efficient water and fertilizer usages).

A traditional method to deliver the collected data is via ground multi-hop relay nodes, which receive and forward data packets via long-distance wireless links. Nevertheless, this method will induce excessive usage of relay nodes and severely impair the network lifetime. The inevitable packet losses caused by unreliable wireless links also reduce the quality of data transmission.

Recent advances in Unmanned Aerial Vehicles (UAVs) have provided a promising paradigm for data collection in large-scale areas due to their benefits of high mobility, flexible maneuverability and lightweight payloads. As the UAVs can fly close to each sensor to collect data via the line-of-sight (LoS) wireless link, sensors can transmit data with a lower transmitting power to extend the battery endurance. However, considering that the UAV's battery capacity limits the cruising duration, and it is costly and inflexible to replace or recharge the ground sensors in vast tracts of farmland, energy conservation is thus a critical issue for both UAV and ground sensors.

To prolong the lifetime of UAV-enabled IoT networks, there are a number of efforts to optimize the UAV's trajectory so as to effectively collect sensory data. For example, Kong et al. [2] consider the trade-off between trajectory length and data collection time in order to minimize one UAV's power cost. Luo et al. [3] focus on minimizing the flight time of multiple UAVs for energy conservation. Ghorbel et al. [4] aim at minimizing the UAV's and sensors' energy consumption by iteratively clustering sensors and optimizing the collection point of each cluster. However, the mentioned works only consider the energy minimization problem, ignoring the quality of the collected data, which inevitably impacts the efficiency of data acquisition.

Specifically, we observe that there exists an obvious trade-off between the energy consumption and data acquisition efficiency. Ideally, the UAV needs to visit the locations of all ground nodes (GNs) one by one for data collection. As a result,

Manuscript received 29 October 2021; revised 12 April 2022 and 10 April 2023; accepted 2 August 2023. Date of publication 18 August 2023; date of current version 11 April 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62072436 and Grant 61732017. The associate editor coordinating the review of this article and approving it for publication was K. R. Chowdhury. *(Corresponding author: Min Liu.)*

Yuchen Zhu and Zhongcheng Li are with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China, and also with the School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing 100049, China (e-mail: zhuyuchen18z@ict.ac.cn; zcli@ict.ac.cn).

Bo Yang is with the College of Information Engineering, Northwest A&F University, Yangling 712100, China (e-mail: yangbo_010@163.com).

Min Liu is with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China, also with the School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing 100049, China, and also with the Zhongguancun Laboratory, Beijing 100190, China (e-mail: liumin@ict.ac.cn).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TWC.2023.3304900.

Digital Object Identifier 10.1109/TWC.2023.3304900

1536-1276 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF TSP-, TSPN-BASED UAV PATH PLAN AND ECAE

Problems	Optimization Objective	Inputs	Outputs	Coverage Model	Search Space
TSP	Energy consumption of UAV	GN coordinates	Visiting order of GNs	None	Finite
TSPN	Energy consumption of UAV	GN coordinates and disk radius	Visiting points for GNs	Disk model [5]	Infinite
ECAE	Energy consumption of both UAV and GNs	GN coordinates and PRR model	Visiting points for GNs	PRR model (Section IV-A)	Infinite

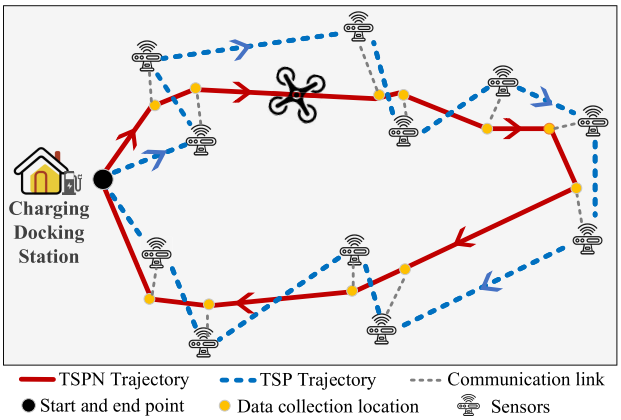


Fig. 1. Comparisons of the TSP and the TSPN trajectories.

the optimal trajectory planning problem can be regarded as the well-known Traveling Salesman Problem (TSP). To further reduce the UAV's flight distance, one assumption is usually made that the UAV can collect data within the transmission coverage ranges of sensors instead of visiting their exact locations. In this case, the problem can be formulated as the Traveling Salesman Problem with Neighborhood (TSPN), which aims to find the shortest tour that visits each sensor's coverage only once. As shown in Fig. 1, the length of TSPN path is obviously shorter than the TSP path.

However, the benefit of TSPN comes at the cost of low data acquisition efficiency. The reason is that TSPN is based on the ideal assumption of *disk coverage model* (i.e., as long as the UAV is within the transmission range of sensors, the UAV can collect data from them with 100% Packet Reception Ratio (PRR)) [5]. In practical scenarios, due to the signal fading and noise interference, corrupted data packets are unavoidable [6]. Typically, a longer distance may cause more corrupted data packets, thus resulting in a lower PRR. Moreover, a lower PRR indicates more re-transmissions for sensor nodes, which will quickly drain the battery.

In this paper, we study the joint optimization of Energy Consumption and data Acquisition Efficiency (ECAE) for UAV-based data collection. There are two major differences among TSP-, TSPN-based UAV path plan and ECAE problem (shown in Table I). First, unlike the TSPN, disk model is not adopted in our work due to the above defects. We employ PRR as a metric to evaluate the quality of data transmission, and strive to explore an effective coverage model to measure the data acquisition efficiency. Second, TSP and TSPN aim to minimize the UAV's path length, i.e., its energy consumption, while the ECAE problem jointly optimizes the energy costs of UAV and low-power GNs. Then, the challenges we face are summarized as follows:

- The PRR is not a simple linear decreasing function of the communication distance. To our knowledge, there is no adequate coverage model to characterize the fine-grained relationship between transmission quality and communication distance.
- The ECAE problem is more complicated than TSP [7] and TSPN [8] (both NP-hard).
- An effective solution for the flight trajectory planning problem should accommodate arbitrary deployments of ground sensors. It is difficult to obtain the optimal trajectory strategy to cover arbitrarily deployed ground sensors within a short finite time.

To date, Deep Reinforcement Learning (DRL) emerges as an effective solution for UAV trajectory optimization [9], [10], [11] due to the benefits of exploring the desired flight strategies by interacting with the environments. In this article, to make fast decisions on flight trajectory in various scenarios (e.g., different numbers and positions of sensors), we propose a DRL-based trajectory optimization scheme named ECAE-MTOD, which includes the Model Training stage and the Optimization & Decision stage. Particularly, we adopt the attention model with deeper network structure to enhance the training efficiency. Our main contributions are as follows:

- Considering the practical signal propagation attenuation and noise interference in wireless transmissions, we derive a mathematical formulation between the PRR and communication distance under the Low-Power Wide Area Network (LPWAN) physical layer protocol – LoRa. Based on this, we propose a PRR-based probabilistic coverage model and set an upper-bounded communication radius according to our theoretical and simulation results.
- To solve the ECAE problem, we propose an attention-based encoder-decoder model to extract the features of ground environment, and train it by using the REINFORCE method with a greedy rollout baseline. We also present an intuitive optimization algorithm to explore the optimal flight trajectory with high-quality data collection.
- We compare our scheme with existing solutions in terms of UAV trajectory length and total energy consumption of both UAV and sensors. Extensive evaluation results demonstrate the superiority of our scheme under different experimental settings.

The rest of this article is organized as follows. Section II reviews the related works. Section III presents the system

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.

model, problem formulation, and communication protocol. Section IV proposes a practical PRR-based coverage model and a novel energy consumption model. Section V presents the ECAE-MTOD scheme, which includes the DRL-based model training as investigated in Section VI, and an intuitive optimization algorithm as investigated in Section VII. Section VIII shows the performance evaluations. Finally, Section IX concludes this article.

Notations: In this article, scalars are denoted by *italic letters*, and vectors are denoted by **boldface letters**. The Euclidean norm of a vector is denoted by $\|\cdot\|$, $\{\cdot\}$ denotes an array and $(\cdot)$ denotes a sequence. $E[\cdot]$ denotes the expectation of a random variable. $A\backslash B \triangleq \{x \in A | x \notin B\}$.

II. Related Work

The literature review has extensively studied data collection methods in IoT networks. Generally, relay-based methods (e.g., tree-based routing [12] and shortest path [13]) deliver data via multiple static relay nodes to the destination. In contrast, UAVs with high mobility are capable of directly collecting data from ground sensors by flying close to them, enabling sensors to transmit data with low power. In view of these advantages, in this section we specifically focus on the potential of UAV for gathering IoT data, and provide a brief review of communication coverage models and trajectory optimization methods relevant to UAV-assisted data collection.

A. Communication Coverage Models

When collecting data from ground wireless nodes, the UAV must be within each node's communication coverage. Therefore, the communication coverage determines the maximum distance between nodes and the UAV during data collection. Existing works have different assumptions on communication coverage models, which can be mainly divided into two categories:

1) Deterministic Coverage Model: The works in [3], [4], and [14] adopt the *disk model*, a classical deterministic coverage model. For the UAV-enabled IoT networks, it indicates that the UAV is certain to receive packets (i.e., PRR=100%) from a GN as long as the UAV is within its coverage (i.e., a circular region centered on this node). However, this assumption is unrealistic since it neglects some practical channel effects (e.g., signal attenuation, channel fading).

2) Probabilistic Coverage Model: Compared with the deterministic model, the probabilistic model is more suitable for practical communication channels. Intuitively, different distances between the UAV and GNs result in different probabilities of successful data reception (i.e., PRR). PRR is distance-dependent and typically goes down with the distance.

Received Signal Strength (RSS)-based probabilistic coverage model [15] is a typical work, which utilizes RSS to measure the link quality. The RSS-based coverage probability is derived in [16] by considering channel fading and shadowing. However, these RSS-based coverage models cannot directly reflect the practical channel quality. The reason is that RSS cannot be used to derive the expected transmission count [17] during one round of data collection. Compared to RSS, PRR is the most intuitive performance indicator of communication quality, and it is directly bound up with the valid amount of collected data. However, obtaining a precise value of PRR typically involves extensive measurements, which is power-intensive and unaffordable for low-power GNs in our case. To address this challenge, we propose a PRR-based coverage model that is derived from **theoretical** analysis of the physical layer design and can estimate PRR based on the distance between the GNs and the UAV. To our knowledge, we are the **first** to investigate the probabilistic coverage model combined with PRR, for evaluating the quality of data transmission in the UAV-assisted data collection.

B. UAV Trajectory Optimization

Existing UAV trajectory optimization methods can be roughly divided into two categories.

1) Conventional Optimization Methods: Many works resort to convex optimization methods by transforming the original non-convex problem into a convex one. For example, Samir et al. [18] formulate the time-constrained trajectory optimization as a mixed-integer non-convex problem and propose a branch-reduce-and-bound (BRB) algorithm to find the global optimal solution. Zeng et al. [19] apply the successive convex approximation (SCA) technique to determine the optimized hovering locations and the visiting order. Zhao et al. [20] study the problem of sum rate maximization in a UAV-assisted NOMA wireless network, and utilize the block coordinate descent (BCD) method to alternately optimize the user scheduling and UAV trajectory. Moreover, Voronoi diagram is often used to find a set of closed UAV paths by selecting and linking Voronoi edges. For example, Baek et al. [21] propose a Voronoi diagram-based UAV trajectory optimization algorithm to maximize the residual energy of sensors. However, the complexities of the above algorithms may grow rapidly with the problem scale. Thus, they may face challenges in making fast decisions particularly in large-scale networks with numerous sensors.

2) Machine Learning (ML) Methods: ML has gained popularity due to its ability to solve complex problems which may be difficult or impossible to solve using conventional optimization methods. Moreover, ML methods can adapt to dynamic environments and provide fast solutions, beneficial for our scenario where some GNs may be added or removed over time. In [22], Bayerlein et al. consider the UAV-enabled data collection within a limited flight duration subject to obstacle avoidance constraints. They propose a novel end-to-end RL approach by training a double deep Q-network (DDQN) to learn the UAV control policy. In [23], to minimize the overall data packet loss, Li et al. propose an on-board deep Q-network (DQN) method to make decisions in power transfer and data collection. Some prior works [24], [25] adopt the age-of-information (AoI) as a metric to optimize the UAV flight trajectory when gathering time-critical information. In particular, Ferdowsi et al. [24] investigate the problem of minimizing the normalized weighted sum of AoI. A long short-term memory (LSTM)-based auto-encoder is proposed to learn the optimal scheduling of status updates

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.

under large-scale scenarios. However, the UAV flight paths considered in [18], [19], [20], [21], [22], [23], [24], and [25] may not necessarily be optimal from the perspective of data acquisition efficiency. The reason is, the communication quality indicators they use (e.g., throughout [19], channel pathloss [4], and data transmission rate [3]) have no direct relevance to the amount of collected qualified data packets, and thus cannot accurately reflect the performance of data collection.

III. System Model and Problem Formulation

A. System Model

Consider that a UAV visits a set of ground nodes (GNs) to collect data from them. These nodes (denoted as $\Phi = \{GN_1, GN_2, \ldots, GN_\phi\}$) are randomly distributed in a 2D field. The coordinate of node GN_i is denoted as $\boldsymbol{w_i} \in \mathbb{R}^{2\times 1}$. We assume that the UAV flies at a fixed altitude H and it starts and ends at the same position whose horizontal coordinate is denoted by $\boldsymbol{l_0} \in \mathbb{R}^{2\times 1}$.

To proceed, we term the location where UAV receives data from GN_i as Data Collection Location (DCL), projected to the plane and denoted by $\boldsymbol{l_i} \in \mathbb{R}^{2\times 1}$, and $\boldsymbol{\mathcal{Q}} = \{\boldsymbol{l_1}, \boldsymbol{l_2}, \ldots, \boldsymbol{l_\phi}\}$. To collect all data, the UAV will successively fly to each GN's DCL. We also define a permutation of the GNs $\boldsymbol{\pi} = (\pi_1, \pi_2, \ldots, \pi_\phi), \pi_i \in \{GN_1, \ldots, GN_\phi\}$ to indicate the visiting order of GNs. In other words, the UAV flies to $\boldsymbol{l_0}, \boldsymbol{l_{\pi_1}}, \boldsymbol{l_{\pi_2}}, \ldots, \boldsymbol{l_{\pi_\phi}}, \boldsymbol{l_0}$ one by one in order. The length of the entire UAV trajectory L can be calculated as

$$L = \|l_{\pi_1} - l_0\| + \sum_{i=1}^{\phi-1} \left\|l_{\pi_{i+1}} - l_{\pi_i}\right\| + \left\|l_0 - l_{\pi_\phi}\right\|. \quad (1)$$

Apparently, there exists an upper limit of GN_i's transmission range, beyond which the UAV can hardly transmit or receive any data packet. Therefore, while the UAV is collecting data from GN_i, the required distance constraint is

$$\|\boldsymbol{l_i} - \boldsymbol{w_i}\| \leqslant D_{max}, \forall i = 1, \ldots, \phi, \quad (2)$$

where D_{max} denotes the maximum horizontal communication range (MHCR) of a GN, and its specific value will be discussed in Section IV.

Moreover, the DCL of UAV for GN_i (i.e., $\boldsymbol{l_i}$) must be the point on the flight trajectory that is closest to GN_i, otherwise, for a given trajectory, UAV can collect data from GN_i with a higher PRR by adjusting $\boldsymbol{l_i}$. The resultant constraint is

$$\boldsymbol{l_i} = \arg\min_{\boldsymbol{l_j}} \|\boldsymbol{l_j} - \boldsymbol{w_i}\|, j \in \{1, \ldots, \phi\}, \forall i = 1, \ldots, \phi. \quad (3)$$

B. Problem Formulation

In this article, we aim to jointly optimize the UAV's and the GNs' energy consumption.

For the UAV, its energy consumption consists of two components: the communication related energy and the propulsion energy. Here, we only consider the latter, because the communication energy consumption can be ignored as compared with that of flight [19]. Note that, although the propulsion energy practically depends on the UAV flying speed, acceleration, flying direction and some other factors, in the considered large-scale farmland data collection, flying distance absolutely dominates the UAV energy consumption. Therefore, we adopt a distance-related energy model [26] to calculate the UAV's energy consumption, denoted by E_{uav}. We assume that UAV flies at a constant speed V_f. Let P_f denote the average flight power of UAV. Then E_{uav} can be obtained by $E_{uav} = P_f \cdot \frac{L}{V_f}$.

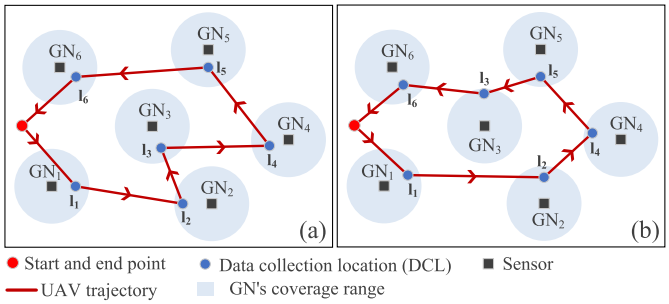


Fig. 2. Once the location of each DCL changes, the visiting order may also change.

For the GNs, we propose a PRR-based energy consumption model in Section IV to estimate the GNs' energy consumption, denoted by E_{gn}. To sum up, the weighted energy consumption sum of the UAV and IoT network, denoted by E_{total}, can be obtained by

$$E_{total} = \omega \cdot E_{gn} + (1 - \omega) \cdot E_{uav}, \quad (4)$$

where ω $(0 \leqslant \omega \leqslant 1)$ is to balance E_{uav} and E_{gn}.

As mentioned earlier, our objective is to minimize the weighted total energy consumption sum of both UAV and GNs, while satisfying the optional scope requirement for DCLs of the ϕ GNs. Given the ϕ GNs, the ECAE problem mainly decides two results: (1) the visiting order $\boldsymbol{\pi}$ of GNs, and (2) the DCL of each GN. Finally, the ECAE problem can be formulated as

$$\min_{\pi, \boldsymbol{\mathcal{Q}}} \quad \omega \cdot E_{iot} + (1 - \omega) \cdot E_{uav}$$
$$s.t. \quad \|\boldsymbol{l_i} - \boldsymbol{w_i}\| \leqslant D_{max}, \forall i = 1, \ldots, \phi, \quad (5)$$
$$\boldsymbol{l_i} = \arg\min_{\boldsymbol{l_j}} \|\boldsymbol{l_j} - \boldsymbol{w_i}\|, j \in \{1, \ldots, \phi\}, \forall i = 1, \ldots, \phi. \quad (6)$$

Note that the ECAE problem is not a simple visiting order design of DCLs. To jointly optimize the total energy consumption and data acquisition efficiency, the choices of DCLs and the visiting order of GNs will influence each other. Specifically, to guarantee a high PRR, each DCL must be selected from the continuous region near the corresponding GN. And different locations for each DCL may induce different visiting orders. For example, as shown in Fig. 2(a), the visiting order of GNs is $\boldsymbol{\pi} = \{GN_1, GN_2, GN_3, GN_4, GN_5, GN_6\}$. An obvious improvement of our objective is shown in Fig. 2(b). As the location of each DCL changes, the visiting order becomes $\boldsymbol{\pi} = \{GN_1, GN_2, GN_4, GN_5, GN_3, GN_6\}$. In brief, it is difficult to obtain the optimal solution when jointly optimizing the visiting order and the DCLs of GNs.

Theorem 1: The ECAE problem is NP-hard.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.

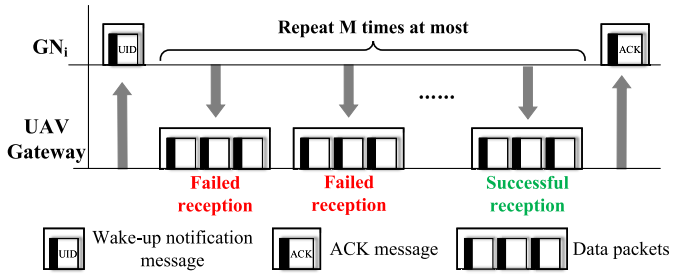


Fig. 3. Communication protocol for UAV-based data collection.

Proof. If we expect the optimal data acquisition performance for each GN, i.e., let $\boldsymbol{l_i} = \boldsymbol{w_i}, \forall i = 1, 2, \ldots, \phi$ to ensure 100% PRR, then the ECAE problem can be reduced to the well-known TSP problem, which has been proven NP-hard [7]. Since a special case of the ECAE problem is NP-hard, the ECAE problem is also NP-hard.

C. Communication Protocol

Fig. 3 illustrates the communication protocol in this article. At each DCL, the UAV broadcasts a wake-up notification message, accompanied by a unique identifier (UID) of a certain GN. After receiving the notification message, the GN corresponding to this UID will immediately reply to the UAV with its local data. Then the GN will wait for an acknowledge (ACK) message from the UAV. Meanwhile, the GN will retransmit its data packets once exceeding the predetermined maximum waiting time. Due to channel burstiness [27], the wireless channel's delivery quality fluctuates between good and poor. Therefore, if a GN has transmitted packets several times but the UAV fails to receive them, there is a higher likelihood that subsequent packets will fail to be received. To avoid GNs wasting too much power on retransmissions, we regulate that each GN can only transmit M[1] times at most.

The sensory data stored in GNs includes temperature, humidity, etc. Unlike image or audio data, these mentioned data can be packaged into a packet and uploaded. The overall transmission process is short. Thus, the UAV can collect data during flying instead of hovering on each DCL.

IV. PRR-Based Coverage Model

In our design, the UAV and the GNs adopt LoRa as their wireless communication protocol. Considering the physical layer (PHY) of LoRa, we derive a novel probabilistic coverage model in the form of PRR (Section IV-A). Based on this coverage model, we propose a PRR-based energy consumption model for the GNs (Section IV-B).

A. PRR-Based Coverage Model for LoRa

1) LoRa Primer: LoRa enables a long-range and low-power communication, and thus is widely adopted by low-power IoT devices [28]. Fig. 4 illustrates the PHY layer mechanism of LoRa. The transmitter performs whitening, hamming coding, interleaving, and gray mapping on the payload bits. The payload bits are then converted into channel bits. After that, by using the Chirp Spread Spectrum (CSS) method, the transmitter modulates channel bits into multiple chirps, whose frequencies change linearly over time.

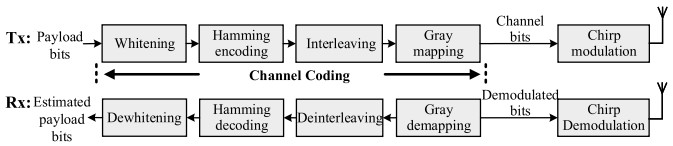


Fig. 4. The PHY layer mechanism in LoRa.

2) PRR Estimation Model for LoRa: Next, by carefully taking the LoRa PHY layer into consideration, we rigorously derive the theoretical relation between the PRR of LoRa and the communication distance. Then we corroborate its accuracy through simulations.

We adopt the free-space channel model. This is reasonable since the UAV can fly close to sensors to establish an unobstructed LOS communication link. Apparently, the movement of the UAV will incur an inevitable Doppler effect and introduce the frequency shift to the LoRa signal, which may degrade the signal quality. However, we point out that the Doppler effect can be naturally compensated by its chirp spreading method. Therefore, the Doppler effect can be overlooked when we estimate the PRR of LoRa.

Given the Friisi' equation [29], the value of SNR, denoted as Γ, can be calculated by

$$\Gamma = \frac{P_t G_t G_r \lambda^2}{\sigma^2 (4\pi)^2 d^2 L_{loss}}, \tag{7}$$

where P_t is the transmission power, d is the Euclidean distance between UAV and GN, L_{loss} denotes miscellaneous losses (transmission lines, antenna losses, etc), G_r is the receiver antenna gain, G_t is the transmitter antenna gain, σ^2 is the additive white Gaussian noise (AWGN) power and λ is the wavelength.

Based on Eq. (7), we then calculate the chirp symbol error ratio (SER). From [30], the closed-form approximation for SER (i.e., P_s) in LoRa under the AWGN channel can be calculated as

$$P_s \approx Q\left(\frac{\sqrt{\Gamma \cdot 2^{SF}} - \left(\left(H_{2^{SF}-1}\right)^2 - \frac{\pi^2}{12}\right)^{1/4}}{\sqrt{H_{2^{SF}-1} - \left(\left(H_{2^{SF}-1}\right)^2 - \frac{\pi^2}{12}\right)^{1/2} + 0.5}}\right), \tag{8}$$

where $Q(\cdot)$ denotes the Q-function and $H_n = \sum_{k=1}^{n} \frac{1}{k}$ denotes the n-th harmonic number. In LoRa, $SF \in \{7, \ldots, 12\}$ is called *spreading factor*.[2] Finally, the SER of channel bits is derived.

As shown in Fig. 4, the LoRa transmitter performs channel coding on payload bits, including whitening, Hamming encoding, interleaving and Gray mapping. To estimate PRR, we need to carefully analyze the effect of these processes:

- **Gray mapping.** In LoRa, there are 2^{SF} chirp symbols to denote SF channel bits. At the receiver, one chirp symbol error may cause one or more bit errors ($[1, SF]$) during Gray demapping. The most likely symbol errors

[1]The value of M should be set to a level that ensures reliable data transmission (e.g., the successful probability within M transmissions is no less than 99.9%) while minimizing the number of retransmissions to save the energy cost of GNs.

[2]Every SF consecutive channel bits are modulated as a chirp symbol.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SIMULATION PARAMETERS

Parameters	Description	Value
P_t	Transmission power of LoRa module [dBm]	10 [32]
σ^2	Noise power [dBm]	-110 [33]
G_t	Transmitter antenna gain [dBi]	4.5 [34]
G_r	Receiver antenna gain [dBi]	4.5 [34]
L_{loss}	Miscellaneous losses [dB]	5
λ	Wavelength [m]	0.33

are the cases where a symbol is confused with one of its closest symbols. Gray mapping makes sure that those most alike symbol errors cause only one bit error, and thus reduce the bit errors induced by one symbol error on average. The Bit Error Ratio (BER), denoted by P_b, can be calculated as [31]

$$P_b = \frac{2^{SF-1}}{2^{SF}-1} P_s \tag{9}$$

In LoRa, SF is no less than 7, thus, P_b can also be approximated as $0.5 \times P_s$.

- **Interleaving** and **whitening**. Interleaving is used to scatter the individual bits of each codeword in time, while whitening is a method to convert payload bits into a pseudo random binary sequence. Interleaving and whitening aim at providing good immunity to noise burst. However, they do not provide any ability of bit error correction for LoRa, and thus they can be neglected when we estimate PRR under the AWGN channel.
- **Hamming encoding.** Hamming encoding has a significant effect on PRR. As LoRa uses (a, n) Hamming codes (the data word length is $a = 4$ and the codeword length is $n \in \{5, 6, 7, 8\}$) for Forward Error Correction (FEC), which offers at most 1-bit error correction capability, thus, LoRa can use the Code Rates (CR) of 4/5, 4/6, 4/7 and 4/8. The codeword error rate (CWER), denoted by P_{cw}, can be calculated as

$$P_{cw} = 1 - ((1-P_b)^n + \tbinom{n}{1} P_b (1-P_b)^{n-1}). \tag{10}$$

Finally, the expression of PRR can be written as

$$PRR \approx 1 - (1 - P_{cw})^{\frac{N_{pl} \cdot SF}{n}}, \tag{11}$$

where N_{pl} denotes the number of chirp symbols in a LoRa packet ($N_{pl} \cdot SF$ denotes the number of payload bits). By construction, N_{pl} is an integer multiple of n and $\frac{N_{pl} SF}{n}$ is the number of codewords in the payload. Eventually, by plugging Eq. (7) into Eq. (11) and replacing Γ with d, the functional relation between PRR and distance can be deduced.

To validate our PRR model, a MATLAB simulator for LoRa signal transmitter/receiver is implemented to measure the PRR for all possible $SF \in \{7, \ldots, 12\}$ with 200 rounds of Monte-Carlo tests. Some parameter settings are listed in Table II. Moreover, we fix the sampling frequency of each LoRa module as 10 MHz, the carrier frequency as 921.5 MHz [32], and the bandwidth as 125 KHz [33]. The data is encapsulated into packets of 160 bits and code rate of 4/5 is adopted in our settings. Fig. 5 displays the comparisons of the analytical results given by Eq. (11) and experimental measurements.

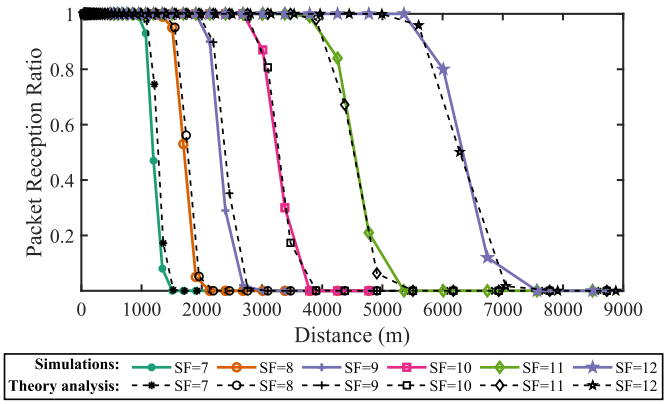


Fig. 5. The PRR of the LoRa modulation under AWGN for all supported spreading factors $SF \in \{7, \ldots, 12\}$.

We observe that, for all possible SFs, simulation results are consistent with the theoretical approximations. Our results also clearly show that neither the deterministic coverage models nor existing probabilistic coverage models can accurately describe the characteristics of wireless transmission. The PRR is not a simple linear function of the distance. Specifically, the PRR is very close to 100% when the distance between the UAV and a GN is relatively short (e.g., when $SF = 7$, the PRR is 99% if the distance is 952.7 m). However, the PRR falls sharply in a certain interval as the distance increases, after which it approaches zero. Based on this, we can set the threshold of PRR (e.g., PRR is no less than 99%) to define the maximum coverage distance for ground sensors (see Section VIII).

B. PRR-Based Energy Consumption Model

The above PRR-based coverage model reveals a fine-grained relation between PRR and distance under LoRa. It also provides an innovative insight to present a novel and practical energy consumption model for GNs.

Foremost, we want to answer the following question: why does the PRR directly determine the GNs' energy consumption. If the channel quality is poor (i.e., low PRR), the GN needs to transmit packets for multiple times before receiving the ACK from the UAV. Apparently, the energy consumption is directly proportional to the number of packet transmissions. Moreover, the GNs' transmitting power is several orders of magnitude greater than their receiving power,[3] we thus mainly care about the transmission consumption.

Next, we think about another question: how to evaluate the transmission consumption of the GNs. We assume that all the GNs have the same transmitting power and data volume ready to be sent. Hence, the transmission consumption depends on the number of re-transmissions caused by failed data reception, which is also the reciprocal of the PRR.

Accordingly, our proposed PRR-based energy consumption model for the GNs is summarized as follows. For any GN_i, $d_i = \sqrt{H^2 + \|\boldsymbol{l_i} - \boldsymbol{w_i}\|^2}$ denotes the distance between the

[3]For example, the GN can use Channel Activity Detection mechanism [35] in LoRa to energy-efficiently receive the signals (e.g., wake-up or ACK) from the UAV.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.

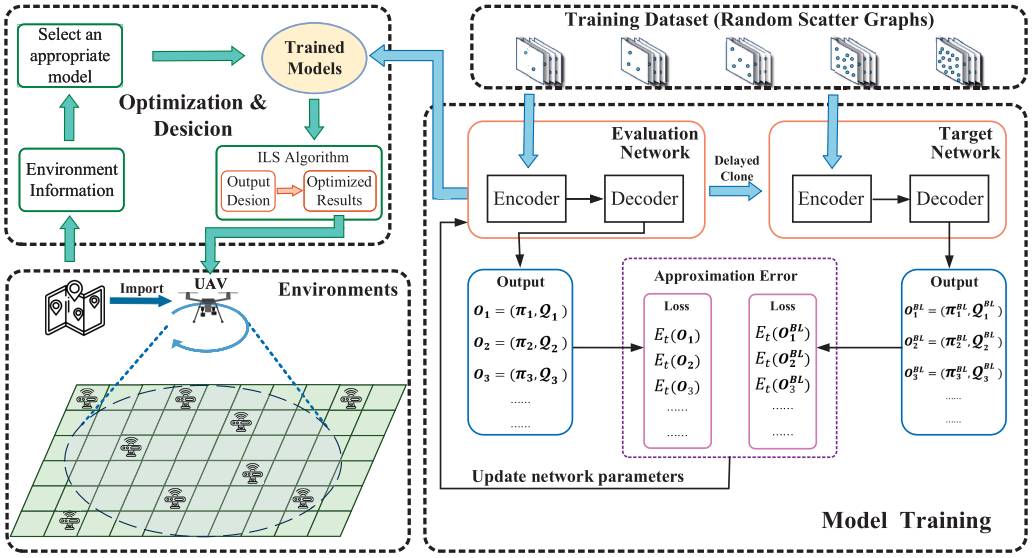


Fig. 6. Framework of the proposed ECAE-MTOD scheme.

GN_i and the UAV when gathering data from GN_i. Let p_i denote the corresponding PRR when the UAV is at $\boldsymbol{l_i}$. Then p_i can be calculated by utilizing d_i and the proposed PRR-based coverage model depicted in Section IV-A.

Let Y_i denote the number of GN_i's transmission until its data is successfully received. Note that each GN can only transmit M times at most, i.e., $1 \leqslant Y_i \leqslant M$ ($Y_i \in \mathbb{Z}$). Then the expectation of Y_i is $E[Y_i] = M \cdot (1-p_i)^{M-1} + \sum_{j=1}^{M-1} (1-p_i)^{j-1} \cdot p_i \cdot j$, which can be simplified to $E[Y_i] = \frac{1-(1-p_i)^M}{p_i}$. And let $Y = \sum_{i=1}^{\phi} Y_i$ denote the total transmitting number of all the GNs. The expectation of Y is $E[Y] = \sum_{i=1}^{\phi} E[Y_i] = \sum_{i=1}^{\phi} \frac{1-(1-p_i)^M}{p_i}$.

We assume that GN_i's energy cost for each data transmission is constant, denoted by W_t. Then the average power consumption of all GNs is $E[E_{gn}] = W_t \cdot \sum_{i=1}^{\phi} \frac{1-(1-p_i)^M}{p_i}$. Eventually, replacing E_{gn} by $E[E_{gn}]$, E_{total} can be reformulated as

$$E_{total} = \omega \cdot W_t \cdot \sum_{i=1}^{\phi} \frac{1-(1-p_i)^M}{p_i} + (1-\omega) \cdot P_f \cdot \frac{\|l_{\pi_1} - l_0\| + \sum_{j=1}^{\phi-1} \|l_{\pi_{j+1}} - l_{\pi_j}\| + \|l_0 - l_{\pi_\phi}\|}{V_f}. \tag{12}$$

V. Design Overview

Given an instance as a graph with ϕ GNs, we can calculate E_{total} of a potential solution with ϕ DCLs and a fixed travelling sequence. However, the optimal solution to minimize E_{total} is difficult to obtain because the search space is very huge: A DCL can be distributed anywhere within a GN's transmission range, let alone the $\phi!$ possible travelling sequences. What is worse, as the designer of a UAV trajectory scheduling scheme, we are unaware of how each user deploys sensors in practice. Thus, it is challenging to keep an acceptable calculation delay and provide fast responses to different situations.

In this work, we explore a DRL-based framework named **ECAE-MTOD** to solve our problem. As shown in Fig. 6, the framework includes two stages: (i) model training; (ii) optimization and decision. In the first stage, we propose an attention-based encoder-decoder model and train it by the REINFORCE algorithm with a deterministic greedy baseline. In the second stage, the UAV selects an appropriate well-trained model to make quick decisions based on practical acquisition scenarios. Then it adopts a fast intuitive optimization algorithm to further improve the decisions.

Note that in the ECAE-MTOD framework, model training can be performed at the UAV control center with powerful computing and processing capabilities. Then the well-trained models and the intuitive optimization algorithm can be deployed and implemented on the UAV. Once users import environment information into UAV or modify some settings (e.g., some GNs out of battery are removed), the UAV can quickly generate an optimal flight trajectory.

VI. Deep Reinforcement Learning for UAV Trajectory Planning

In this section, we first describe the attention-based model in detail and then show how to train it by using the REINFORCE algorithm with a simple but effective baseline.

A. Attention-Based Encoder-Decoder Network Model

1) Attention Mechanism for Feature Extraction: Attention mechanism is originally motivated by how we pay visual attention to different regions of an image. In the deep learning community, attention recently has been a popular concept and a useful tool [36]. Briefly, attention can be broadly interpreted as a vector of importance weights: it allows the model to attend for the most relevant parts of the input, thus yielding improvements in the quality of the generated outputs.

The reasons to choose the attention model are as follows: (i) Attention model reduces the computational cost of processing high-dimensional inputs by only selecting the subsets of

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.

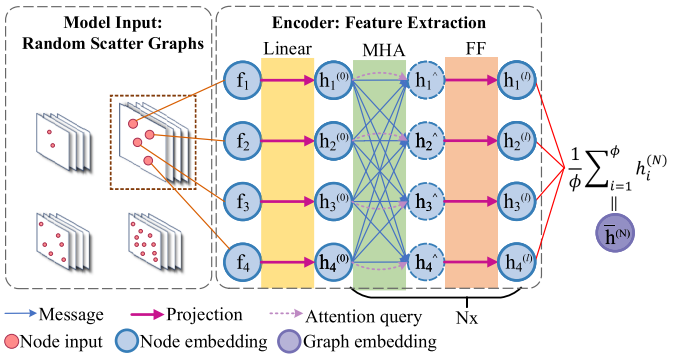


Fig. 7. Attention-based encoder (take four node inputs as an example).

inputs. (ii) The attention layers (see Appendix A) can effectively capture the local dependencies and extract important features, e.g., network features like the graphical layout. (iii) The attention-based encoder-decoder models normally outperform other encoder-decoder models when dealing with long sequences. (iv) Some existing works (e.g., [37], [38]) have proven that the attention-based models can effectively solve a series of combinatorial optimization problems that search the optimal object from a finite set of objects. Our problem does not fall into any standard combinatorial optimization problem, but shares some similarities with them. As mentioned in Section III-B, the ECAE problem can be split into two tasks: One is a classification task that outputs a permutation of nodes, which is a combinatorial optimization problem; the other is a regression task which selects ϕ DCLs from the continuous 2D space.

2) Encoder: Fig. 7 shows the architecture of our encoder, which adapts from the Graph Attention Network [39] and considers the input as a graph structure regardless of the order of inputs. It is beneficial since whatever the input order of GNs is, the output is unchanged as long as the GNs' coordinates are unchanged. In our design, we define a problem instances s as a fully connected graph with ϕ nodes (i.e., GNs). Each node is denoted by i $(i = 1, \cdots, \phi)$, and each node i has a 2-dimensional feature vector $\boldsymbol{f_i} = [x_i, y_i]$, which denotes its coordinate. The inputs to the encoder are each node's $\boldsymbol{f_i}$. We also define a tuple $\boldsymbol{O} = (\boldsymbol{\pi}, \boldsymbol{\mathcal{Q}})$ as a feasible solution of the instance s. As mentioned in Section III-A, we use $\boldsymbol{\pi} = (\pi_1, \pi_2, \ldots, \pi_\phi)$ as a permutation of the nodes. At each step τ, $\pi_\tau \in \{1, \ldots, \phi\}$ where each node i appears only once. $\boldsymbol{\mathcal{Q}} = \{\boldsymbol{l_1}, \ldots, \boldsymbol{l_\phi}\}$ is a set of DCLs. We add a known position $\boldsymbol{l_0} \in \mathbb{R}^{2\times 1}$ as the start and end point.

We strive to learn the parameters $\boldsymbol{\theta}$ of a stochastic policy $p_{\boldsymbol{\theta}}(\boldsymbol{O}|s)$ for selecting a solution $\boldsymbol{O}$ given a problem instance s. It can be factorized and parameterized by $\boldsymbol{\theta}$ as

$$p_{\boldsymbol{\theta}}(\boldsymbol{O}|s) = \prod_{\tau=1}^{\phi} p_{\boldsymbol{\theta}}(\pi_\tau, \boldsymbol{l}_{\boldsymbol{\pi}_\tau}|s, \boldsymbol{\pi_{1:\tau-1}}, \boldsymbol{l}_{\boldsymbol{\pi_{1:\pi_{\tau-1}}}}). \quad (13)$$

We denote the output of the layer $\ell \in \{0, 1, \ldots N\}$ as $\boldsymbol{h_i^{(\ell)}}$. As shown in Fig. 7, each $\boldsymbol{f_i}$ is first projected to a d_h-dimensional node embedding $\boldsymbol{h_i^{(0)}}$ (we use d_h=128) through a linear transformation with parameters $\boldsymbol{W^0}$ and $\boldsymbol{b^0}$: $\boldsymbol{h_i^{(0)}} = \boldsymbol{W^0 f_i} + \boldsymbol{b^0}$. Next, the node embeddings are further processed by N_x (we use $N_x = 3$) sequential layers, each layer involves two sublayers: a multi-head attention (MHA) sublayer [40] and a node-wise feed-forward (FF) sublayer.

Intuitively, the MHA sublayer (see Appendix B) can be viewed as a weighted message exchange algorithm among all the nodes in a graph. The MHA sublayer aggregates N_M (we choose $N_M = 8$) concurrent and independent attention layers. Every attention layer calculates the importance of each pair of nodes and fuses the relationship matrix into each node embedding.

The FF sublayer (see Appendix C) is to make network deeper and more complex. The encoder can thus capture not only some straightforward features (e.g., GNs' positions), but underlying features (e.g., the importance of messages from distant and nearby neighbors) of the model input. We also insert a skip-connection [41] and batch normalization operation [42] after each sublayer.

The encoder will output the final node embeddings (denoted as $\boldsymbol{h_i^{(N)}}$) of all ϕ GNs. We then average all node embeddings as a **graph embedding**, denoted as $\overline{\boldsymbol{h}}^{(\boldsymbol{N})}$. Both the graph embedding $\overline{\boldsymbol{h}}^{(\boldsymbol{N})}$ and ϕ node embeddings $\boldsymbol{h_i^{(N)}}$ are used as the inputs to the following decoder.

3) Decoder: The task of the decoder is to output the visiting order of all GNs, as well as the corresponding DCLs. Specifically, at each step τ, $\tau = 1, \ldots, \phi$, the decoder outputs a node index π_τ and a DCL $\boldsymbol{l}_{\boldsymbol{\pi}_\tau}$ based on the embeddings produced by the encoder and the outputs generated ahead of time τ. Apparently, the decoder needs to perform two relevant tasks simultaneously:

- Deciding π_τ is a classification task since each node's index corresponds to a category label.
- Finding $\boldsymbol{l}_{\boldsymbol{\pi}_\tau}$ from the 2D space is a regression task (make predictions for continuous values).

Classification task: The classification task is to decide which node to visit at each step. At step τ, this part of decoder (we call it *classification decoder* for distinction) takes as input the node embeddings $\boldsymbol{h_i^{(N)}}$, graph embedding $\overline{\boldsymbol{h}}^{(\boldsymbol{N})}$, and a special context embedding $\boldsymbol{CT_{(c)}^{(\tau)}}$.

- *Classification Context Embedding.* We construct a classification context embedding to represent the current state. Note that, it is safe to neglect previous visiting order of the GNs prior to the step τ, as well as the corresponding DCLs. The reason is that the decision only depends on the current state (i.e., Markov property). The previous outputs of the decoder can construct a partial visiting path. The remaining problem becomes to find a visiting path, starting from the last DCL, to optimize the ECAE objective. Thus, we explicitly discard outputs prior to τ, only mapping the graph embedding, the embeddings of the first node π_1, the last node $\pi_{\tau-1}$, and their DCLs to the context. Fig. 8 shows an example where nodes 1, 3 and their DCLs $\boldsymbol{l_1}$, $\boldsymbol{l_3}$ have been output at steps 1 and 2. For the 2-dimension DCL $\boldsymbol{l_i}$, we calculate its d_h-dimension *location embeddings* $\boldsymbol{g_i}$ through a FF layer. Then a $(5 \cdot d_h)$-dimensional classification context

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.

embedding $\boldsymbol{CT}_{(c)}^{(\tau)}$ is constructed as follows:

$$\boldsymbol{CT}_{(c)}^{(\tau)} = \begin{cases} [\bar{\boldsymbol{h}}^{(N)}, \boldsymbol{h}_{\pi_1}^{(N)}, \boldsymbol{h}_{\pi_{\tau-1}}^{(N)}, \boldsymbol{g}_{\pi_1}, \boldsymbol{g}_{\pi_{\tau-1}}], \tau > 1, \\ [\bar{\boldsymbol{h}}^{(N)}, \boldsymbol{v}^{\boldsymbol{l}}, \boldsymbol{v}^{\boldsymbol{f}}, \boldsymbol{g}^{\boldsymbol{l}}, \boldsymbol{g}^{\boldsymbol{f}}], \tau = 1, \end{cases} \tag{14}$$

where $[\cdot,\cdot,\cdot,\cdot,\cdot]$ is the horizontal concatenation operator to put the five embeddings together. When $\tau = 1$, we use the learned 128-dimension parameters $\boldsymbol{v}^{\boldsymbol{l}}, \boldsymbol{v}^{\boldsymbol{f}}, \boldsymbol{g}^{\boldsymbol{l}}$ and $\boldsymbol{g}^{\boldsymbol{f}}$ as placeholders.

Then $\boldsymbol{CT}_{(c)}^{(\tau)}$ and node embeddings $\boldsymbol{h}_i^{(N)}$ are further refined and fused by an MHA layer for a new context embedding $\boldsymbol{CT}_{(c)}^{(\tau,2)}$. Particularly, for each head we calculate query vector $\boldsymbol{q}_{(c)}$ only using $\boldsymbol{CT}_{(c)}^{(\tau)}$. And keys with d_k-dimension and values are from $\boldsymbol{h}_i^{(N)}$:

$$\boldsymbol{q}_{(c)} = \boldsymbol{W}^{\boldsymbol{Q}}\boldsymbol{CT}_{(c)}^{(\tau)}, \boldsymbol{k_i} = \boldsymbol{W}^{\boldsymbol{K}}\boldsymbol{h}_i^{(N)}, \boldsymbol{v_i} = \boldsymbol{W}^{\boldsymbol{V}}\boldsymbol{h}_i^{(N)}. \tag{15}$$

This is to quantify the relative significance of each node embedding to $\boldsymbol{CT}_{(c)}^{(\tau)}$. With $\boldsymbol{q}_{(c)}$, $\boldsymbol{k_i}$ and $\boldsymbol{v_i}$, we calculate the outputs of each head separately by using Eq. (19). As in [43], we mask nodes already visited before time τ by setting $\frac{\boldsymbol{q}_{(c)} \cdot \boldsymbol{k}_j{}^T}{\sqrt{d_k}} = -\infty, j = \pi_{\tau'}, \forall \tau' < \tau$.

- *Calculation of Output Probabilities.* Given node embeddings and a state representation (a query vector $\boldsymbol{q}_{(c)}^{\boldsymbol{2}}$ from context embedding $\boldsymbol{CT}_{(c)}^{(\tau,2)}$), the classification decoder first calculates the log-probabilities (logits) with a single-head attention layer. Also, we mask nodes already visited before time τ by setting their corresponding logits as $-\infty$, which ensures that our classification decoder outputs a valid order of the input nodes. Moreover, we clip the other log-probabilities in $[-C, C]$ (we choose $C = 10$), and then calculate the final output probabilities of the potential nodes by a softmax:

$$\begin{aligned} p_i &= p_\theta(\pi_\tau = i | s, \boldsymbol{\pi_{1:\tau-1}}, \boldsymbol{l}_{\pi_1:\pi_{\tau-1}}) \\ &= softmax(C \cdot \tanh(\frac{\boldsymbol{q}_{(c)}^{\boldsymbol{2}} \cdot \boldsymbol{k}_j{}^{\boldsymbol{T}}}{\sqrt{d_k}})), \end{aligned} \tag{16}$$

where $\frac{\boldsymbol{q}_{(c)}^{\boldsymbol{2}} \cdot \boldsymbol{k}_j{}^{\boldsymbol{T}}}{\sqrt{d_k}} = -\infty$ if node j has been output.

Regression task: The regression task is to predict the corresponding DCL's coordinate based on the node index output by the classification decoder at each step. At time τ, the regression decoder takes as input the node embeddings $\boldsymbol{h}_i^{(N)}$, graph embedding $\bar{\boldsymbol{h}}^{(N)}$, a regression context embedding $\boldsymbol{RT}_{(c)}^{(\tau)}$, and an output node index π_τ from the classification decoder.

- *Regression Context Embedding.* We also augment a regression context embedding to represent the current state for the regression task. At step τ, the regression context embedding $\boldsymbol{RT}_{(c)}^{(\tau)}$ is a concatenation of $\bar{\boldsymbol{h}}^{(N)}$, the node embeddings and location embeddings of the first and last predicted DCLs. Differently, considering the following repetitive concatenations, we add $\boldsymbol{h_1}^{(N)}$ and $\boldsymbol{h_{\tau-1}}^{(N)}$, $\boldsymbol{g_1}$ and $\boldsymbol{g_{\tau-1}}$ respectively, to avoid vectors with high dimension. Then we concatenate the results and $\bar{\boldsymbol{h}}^{(N)}$ to produce a $(3 \cdot d_h)$-dimension $\boldsymbol{RT}_{(c)}^{(\tau)}$.
- *Relative Deviation Vector.* As shown in Fig. 8, the regression decoder feeds node embeddings $\boldsymbol{h}_i^{(N)}$ into a FF layer for d_h-dimension vectors $\boldsymbol{h}_i'$. Then we concatenate each $\boldsymbol{h}_i'$ and $\boldsymbol{RT}_{(c)}^{(\tau)}$ to get $(4 \cdot d_h)$-dimension vectors as the inputs of a three-layer fully-connected network. Accordingly, the outputs of each layer are 64-dimension, 32-dimension and 2-dimension vectors, respectively. We write the final 2-dimension result vectors as $\boldsymbol{RD_i}^{(\tau)} = [\Delta {x_i}^{(\tau)}, \Delta {y_i}^{(\tau)}]$, named *Relative Deviation Vectors*, denoting the relative deviations between the predicted DCLs and the corresponding node positions. Note that $\boldsymbol{l_{\pi_\tau}}$ should be within the coverage of node π_τ. If $\left\|\Delta {x_{\pi_\tau}}^{(\tau)} + \Delta {y_{\pi_\tau}}^{(\tau)}\right\|$ is beyond the MHCR D_{max}, $\Delta {x_{\pi_\tau}}^{(\tau)}$ or $\Delta {y_{\pi_\tau}}^{(\tau)}$ will be cut off at D_{max} for simplicity. In Fig. 8, at step $\tau = 3$, given $\pi_3 = 2$ from the classification decoder, the output of regression decoder is $\boldsymbol{l_{\pi_3}} = [x_2 + \Delta {x_2}^{(3)}, y_2 + \Delta {y_2}^{(3)}]$.

B. Reinforce Learning Training Method With Greedy Rollout Baseline

Section VI-A introduces our model and defines a probability distribution $p_{\boldsymbol{\theta}}(\boldsymbol{O}|s)$ under an instance s, from which we can sample to get a trajectory. In the ECAE problem, the UAV works as an agent, periodically observing its surrounding environment and performing its action based on the current policy parameter $\boldsymbol{\theta}$. To find a good trajectory for the UAV, we need to obtain the optimal model parameter $\boldsymbol{\theta}^*$ that can be trained from samples. By convention, we define the state, action, reward (loss) and training of our proposed DRL algorithm.

1) State: At each step τ, the state consists of three parts:

- The index sequence of GNs whose data have been collected before step τ;
- The coordinates of DCLs corresponding to GNs that have been visited;
- The locations of remaining unvisited GNs.

In fact, the classification context embeddiing $\boldsymbol{CT}_{(c)}^{(\tau)}$ and the regression context embedding $\boldsymbol{RT}_{(c)}^{(\tau)}$ in the decoder represent the state at step τ.

2) Action: At each step τ, the action consists of two parts:

- The index of GN to be visited next, which is the output of the classification decoder;
- The coordinate of DCL corresponding to the next GN to be visited, which is the output of the regression decoder.

3) Reward (Loss): We use $E_t(\boldsymbol{O})$ to represent our objective value calculated by Eq. (12) for a feasible solution $\boldsymbol{O}$. And we define the loss $\mathcal{L}(\boldsymbol{\theta}|s) = E_{p_{\boldsymbol{\theta}}(\boldsymbol{O}|s)}[E_t(\boldsymbol{O})]$, that is the expectation of $E_t(\boldsymbol{O})$. For a given s, the main principle is to delegate "good" tours (low loss) with greater probability and "undesirable" tours (high loss) with lower probability.

4) Training: To solve the ECAE problem with continuous state and action space, we adopt the REINFORCE [44] policy gradient algorithm with baseline $b_{\boldsymbol{\theta}^{BL}}(s)$ to optimize $\mathcal{L}$:

$$\nabla\mathcal{L}(\boldsymbol{\theta}|s) = E_{p_{\boldsymbol{\theta}}(\boldsymbol{O}|s)}[(E_t(\boldsymbol{O}) - b_{\boldsymbol{\theta}^{BL}}(s))\nabla \log(p_{\boldsymbol{\theta}}(\boldsymbol{O}|s))]. \tag{17}$$

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.

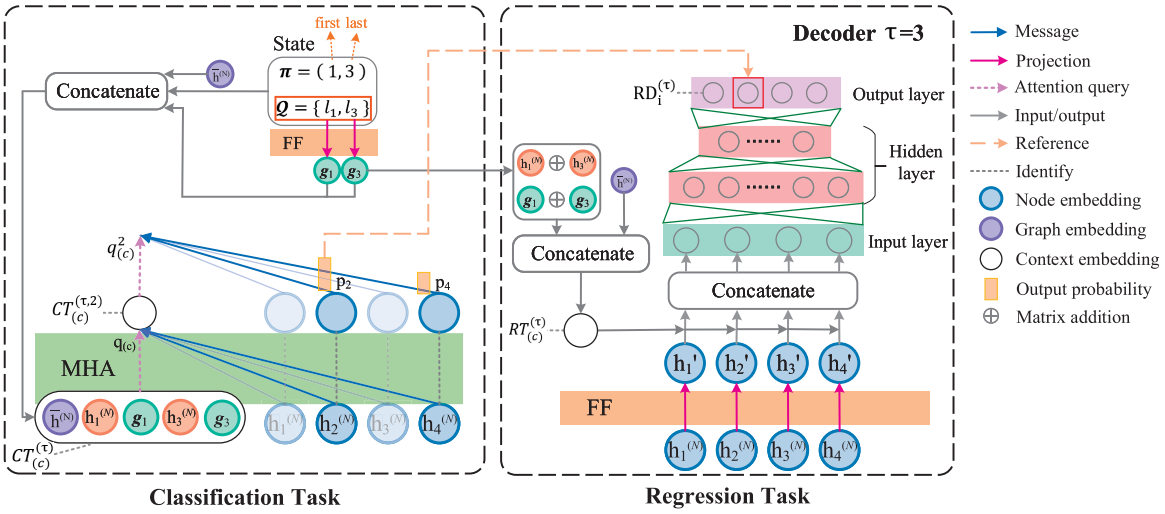


Fig. 8. Attention-based decoder when $\tau = 3$.

REINFORCE relies on an estimated return by Monte-Carlo (MC) methods using episode samples to update the policy parameter $\boldsymbol{\theta}$. Although MC estimates are unbiased, they tend to produce high variance and require excessive sample sizes, resulting in slow convergence. A widely used variation of REINFORCE is to subtract a baseline value from the total return (or loss) to reduce the variance of gradient estimation while keeping the bias unchanged, thus increasing speed of learning.

In this article, we use a rollout baseline similar to that of [37]. Our baseline is defined as the cost of a solution resulting from a policy determined by the best model so far. We keep two sets of networks with the same structure: One is always updated to ensure that it has the latest parameters $\boldsymbol{\theta}$, called *evaluation network*; the other is called the *target network*, usually freezing a set of outdated parameters $\boldsymbol{\theta}^{BL}$ referring to our baseline at each epoch, to assess whether the latest version of evaluation network is better. At the end of each epoch, we substitute $\boldsymbol{\theta}$ for $\boldsymbol{\theta}^{BL}$ only if the improvement is prominent as measured by a paired t-test ($\alpha = 5\%$) on 10000 separate evaluation instances, to ensure that we always reserve the best model till now. Moreover, we use Adam [45] as optimizer because it has high computational efficiency and low memory requirements. The REINFORCE algorithm with the rollout baseline is shown in Algorithm 1.

Algorithm 1 REINFORCE Training

Input: number of epochs E, training steps per epoch T, batch size B, significance α

Randomly initialize $\boldsymbol{\theta}$, $\boldsymbol{\theta^{BL}} \leftarrow \boldsymbol{\theta}$;
for $epoch = 1 : E$ **do**
  for $step = 1 : T$ **do**
    $s_i \leftarrow \text{SampleInstance}(), \forall i \in \{1, \ldots, B\}$;
    $\boldsymbol{O_i} \leftarrow \text{SampleSolution}(p_{\boldsymbol{\theta}}(\cdot|s_i)),$
    $\forall i \in \{1, \ldots, B\}$;
    $\boldsymbol{O_i^{BL}} \leftarrow b_{\boldsymbol{\theta^{BL}}}(s_i), \forall i \in \{1, \ldots, B\}$;
    $\nabla \mathcal{L} \leftarrow$
    $\frac{1}{B} \sum_{i=1}^{B} (E_t(\boldsymbol{O_i}) - E_t(\boldsymbol{O_i^{BL}})) \nabla \log p_\theta(O_i)$;
    $\boldsymbol{\theta} \leftarrow \text{Adam}(\boldsymbol{\theta}, \nabla \mathcal{L})$;
  end
  if $t - test(p_{\boldsymbol{\theta}}, p_{\boldsymbol{\theta^{BL}}}) < \alpha$ **then**
    $\boldsymbol{\theta^{BL}} \leftarrow \boldsymbol{\theta}$.
  end
end
return $\boldsymbol{\theta}^* = \boldsymbol{\theta}$

VII. Intuitive Algorithm for Optimization and Decision

A. Practical Issue

A distinct drawback of the trained models is that the size of input is fixed. If one user actually deploys $\phi - 1$ GNs, it seems infeasible for a trained model with ϕ inputs to deal with this situation. In practice, considering all possible inputs during the training phase is difficult and expensive.

Here, we assume only the models with $10 \times \xi$ ($\xi \in \mathbb{Z}^+$) inputs will be obtained after model training. Firstly, count the number of GNs according to users' actual demands, denoted by $N_g \in \mathbb{N}$. If N_g can be divisible by 10, there exists a model for direct use; if not, the UAV will choose a model with $\left\lceil \frac{N_g}{10} \right\rceil \times 10$ inputs, and the inputs are divided into two parts. Besides N_g node coordinates, the remaining part is $\left\lceil \frac{N_g}{10} \right\rceil \times 10 - N_g$ duplicates of any one of the N_g coordinates, which are essentially meaningless and do not affect the outcome, just for catering to the size of inputs. Finally, the chosen model generates an intial result $\boldsymbol{O} = (\boldsymbol{\pi}, \boldsymbol{\mathcal{Q}})$.

B. Intuitive Optimization Algorithm

The solution space of the ECAE problem is very huge due to its NP-hardness. And the DRL-based model training typically offers a feasible solution rather than an optimal one. Thus, we further optimize the model output by an intuitive optimization algorithm, named Iterative Local Search (ILS) algorithm.

An example is illustrated in Fig. 9, where $\phi = 7$, $\boldsymbol{\pi} = \{GN_1, GN_2, \ldots, GN_7\}$ and $\boldsymbol{\mathcal{Q}} = \{\boldsymbol{l_1}, \boldsymbol{l_2}, \ldots, \boldsymbol{l_7}\}$. The

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.

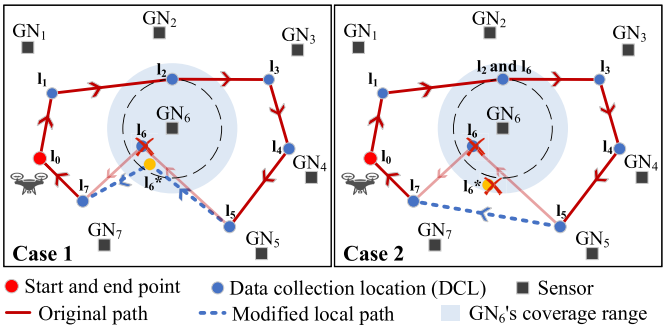


Fig. 9. An example of the proposed iterative local search algorithm.

red solid line represents an original path of UAV. The ILS algorithm produces the optimized trajectory in five steps. In the first step, randomly pick a GN, e.g., GN_6. Find out the DCL closest to GN_6 other than $\boldsymbol{l_6}$, i.e., $\boldsymbol{l_2}$. Secondly, attempt to move $\boldsymbol{l_6}$ to a better place $\boldsymbol{l_6}^*$ within GN_6's coverage such that E_{total} is minimized. This is a standard convex optimization problem, which is easy to solve. Thirdly, depending on the position of $\boldsymbol{l_6}^*$, two cases may happen: if the distance between GN_6 and $\boldsymbol{l_6}^*$ is smaller than that between GN_6 and $\boldsymbol{l_2}$ (case 1 of Fig. 9), we choose $\boldsymbol{l_6}^*$ as the new DCL of GN_6; if not (case 2), we merge $\boldsymbol{l_6}$ and $\boldsymbol{l_2}$, thereby reducing the UAV path length while improving the PRR of GN_6. Fourthly, calculate the current value of E_{total}, and repeat steps 1-4 until the loop count reaches a given value R. Eventually, $\boldsymbol{\mathcal{Q}}$ and $\boldsymbol{\pi}$ are settled. A further improvement is taking $\boldsymbol{\mathcal{Q}}$ as inputs of TSP solvers like Concorde [46]. We present the formal steps of the ILS algorithm in Algorithm 2. The time complexity of Algorithm 2 depends on the adopted TSP solver, which can be solved in $O(n^2 2^n)$ time [47].

Algorithm 2 Iterative Local Search Algorithm

Input: $\boldsymbol{l_0}$, Φ, $\boldsymbol{\pi}$, $\boldsymbol{\mathcal{Q}}$, D_{max}, ω, W_t, P_t, V_f, and M
Output: Updated $\boldsymbol{\pi}$ and $\boldsymbol{\mathcal{Q}}$; the optimized E_{total}

for $r = 1 : R$ **do**
  Step 1: Randomly choose a GN (denoted as GN_i) from ϕ GNs. Assume that its order in $\boldsymbol{\pi}$ is π_i;
  // Denote the DCL closet to GN_i except $\boldsymbol{l_i}$ as $\boldsymbol{l_j}$
  $\boldsymbol{l_j} = \arg\min_{\boldsymbol{l_k} \in \boldsymbol{\mathcal{Q}} \setminus \boldsymbol{l_i}} \|\boldsymbol{l_k} - \boldsymbol{w_i}\|$;
  // Denote the distance between GN_i and $\boldsymbol{l_j}$ as d_{ij}
  $d_{ij} = \|\boldsymbol{l_j} - \boldsymbol{w_i}\|$;
  Step 2: Calculate the path length L by Eq. (1);
  // Move $\boldsymbol{l_i}$ to a better place $\boldsymbol{l_i}^*$ within GN_i's coverage range so that E_{total} is minimized

$$\boldsymbol{l_i}^* = \arg\min_{l_i' \in \mathbb{R}^2} \omega \cdot W_t \cdot \Big(\frac{1-(1-PRR(\|\boldsymbol{l_i}' - \boldsymbol{w_i}\|))^M}{PRR(\|\boldsymbol{l_i}' - \boldsymbol{w_i}\|)} + \sum_{k \in \{1,\dots,\phi\} \setminus i} \frac{1-(1-PRR(\|\boldsymbol{l_k} - \boldsymbol{w_k}\|))^M}{PRR(\|\boldsymbol{l_k} - \boldsymbol{w_k}\|)}\Big) + (1-\omega) \cdot P_f \cdot \Big(\frac{L - (\|\boldsymbol{l_i} - \boldsymbol{l_{\pi_i - 1}}\| + \|\boldsymbol{l_i} - \boldsymbol{l_{\pi_i + 1}}\|)}{V_f} + \frac{\|\boldsymbol{l_i}' - \boldsymbol{l_{\pi_i - 1}}\| + \|\boldsymbol{l_i}' - \boldsymbol{l_{\pi_i - 1}}\|}{V_f}\Big);$$

  subject to $\|\boldsymbol{l_i}' - \boldsymbol{w_i}\| \leqslant D_{\max}$;
  // $PRR(\cdot)$ means the calculated PRR values by using equations in Section IV-B
  Step 3: Analyze the following two cases;
  if $\|\boldsymbol{w_i} - \boldsymbol{l_j}\| \leqslant d_{ij}$ **then**
    $\boldsymbol{l_i} = \boldsymbol{l_i}^*$, and the updated local trajectory is $\boldsymbol{l_{\pi_i - 1}} \rightarrow \boldsymbol{l_i}^* \rightarrow \boldsymbol{l_{\pi_i + 1}}$;
  else
    $\boldsymbol{l_i} = \boldsymbol{l_j}$, and the updated local trajectory is $\boldsymbol{l_{\pi_i - 1}} \rightarrow \boldsymbol{l_{i+1}}$;
  end
  Step 4: Calculate E_{total} by Eq. (12);
end
Step 5: Further shorten the trajectory length by taking $\boldsymbol{\mathcal{Q}}$ as the input of TSP solvers;
return $\boldsymbol{\pi}$, $\boldsymbol{\mathcal{Q}}$ *and* E_{total}

VIII. Performance Evaluation

In this section, extensive simulations are conducted to validate the performance of our proposed ECAE-MTOD scheme.

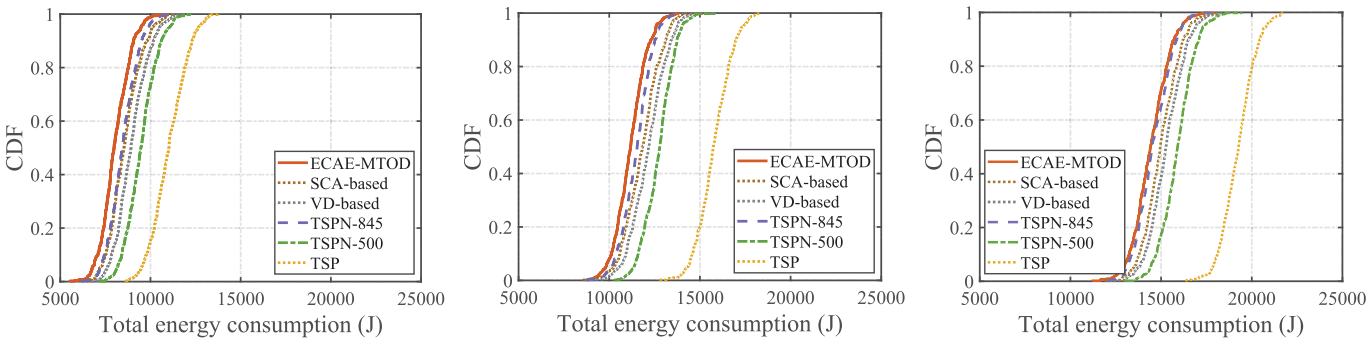
A. Experiment Settings

We consider an area of size $10km \times 10km$, where ϕ GNs are uniformly and randomly placed. Suppose that a UAV[4] starts at a random location and its overall energy is adequate to complete a round trip (e.g. customized multi-purpose UAVs). We perform the model training on an Ubuntu 18.04 server with 4 GPUs Nvidia 1080Ti and Intel Core i7-7700K CPU @ 4.20GHz and 32GB RAM. During training, every epoch we process 2000 batches of 500 instances and we train for 20 epochs by using datasets of randomly generated scatter plots. The proposed ILS algorithm is implemented by MATLAB R2020b on a Windows 10 server with Intel Core i7-10510U CPU @ 2.30GHz and 16GB RAM. We set the maximal iteration $R = 500$ when carrying out the ILS algorithm to optimize the model outputs. The main simulation parameters are listed in Table III.

[4]Though a single UAV is employed in this study, our scheme can be extended to multi-UAV scenarios by partitioning the entire acquisition area into multiple subregions and assigning each UAV to collect data in a specific subregion.

Here, we compare ECAE-MTOD with the SCA-based UAV path planning algorithm in [19] and the Voronoi diagram-based (VD-based) algorithm in [21]. Meanwhile, we consider the TSP and TSPN that are closely relevant to our work, and utilize their solutions [37] and [49] as baselines. It has been proved that the learning-based TSP solution [37] can get close to the optimal results for problems up to 100 nodes. Suppose that the TSP solution can reach 100% PRR since it treats all GNs' positions as DCLs. The TSPN solution [49] determines UAV path while restricting a fixed coverage range of GNs.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.



(a) $\phi = 20$ (b) $\phi = 40$ (c) $\phi = 60$

Fig. 10. CDF of total energy consumption when $\phi = 20, 40$ and 60.

TABLE III
SIMULATION PARAMETERS

Parameters	Description	Value
ϕ	The number of GNs	20 / 40 / 60
H	The UAV altitude [m]	80
V_f	The flight speed of UAV [m/s]	20 [48]
P_f	The average flight power of UAV [W]	600 [48]
W_t	The energy consumption per delivery of GNs [J]	5.73×10^{-4} [19]
M	The maximum times of sending packets for GNs	3

Reference [49] provides a heuristic algorithm within 1% of optimality by using the mixed-integer nonlinear programming (MINLP) solver Bonmin.

Note that *coverage range* means something different in TSPN and ECAE problem. TSPN assumes PRR=100% within coverage and PRR=0 otherwise, while ECAE considers a fine-grained trend of PRR with communication distance. Based on the simulation results in Fig. 5, without loss of generality, we take the curve of PRR with $SF = 7$ as an example to quantify GN's coverage in the TSPN and the ECAE problem. For the TSPN, when the distance between UAV and GN is less than 849m (horizontal distance: $\sqrt{849^2 - 80^2} \approx 845.2$m), PRR keeps 100%. Thus, we consider the MHCR D_{max} for TSPN as 500m and 845.2m, denoted by TSPN-500 and TSPN-845 respectively. For the ECAE problem, we set the corresponding horizontal distance as $D_{\max} = \sqrt{1125^2 - 80^2} \approx 1122.2$m which corresponds to PRR=90%. Thus, UAV can collect data from each GN with a very high probability of no less than $1 - (1 - 0.9)^3 = 99.9\% \approx 100\%$ (up to $M = 3$ retransmissions) after a single flight, eliminating the need for multiple flight trips.

B. Evaluation Results

1) Impact of Node Density on Total Energy Consumption: We fix the scale of acquisition area, and vary the number of GNs (ϕ=20, 40 and 60). In each case, we plot the Cumulative Distribution Function (CDF) of E_{total} through 500 independent trials. Here we set $\omega = 0.99$ due to the obvious gap of energy consumption between the UAV and low-power ground sensors.

Figs. 10(a), (b) and (c) display the CDF of E_{total} under the three cases. As the number of GNs increases, E_{total} is increasing as expected. Obviously, more GNs consume more energy and usually result in a longer flight for the UAV to collect data. From Fig. 10, we can see that the ECAE-MTOD scheme always performs best in saving energy. For example, when ϕ=20, it can reduce the average total energy cost by 27.1% as compared to the TSP solution while remaining the overall PRR close to 100%. As for the SCA-based algorithm, VD-based algorithm, TSPN-845 and TSPN-500, the reduction percentages are 6.6%, 10.1%, 4.9% and 15.1% respectively. The SCA-based algorithm fixes the visiting order of GNs obtained by a TSP solver, which greatly limits the performance. The VD-based algorithm is skilled in generating a feasible trajectory quickly. However, it is easy to fall into local optimum. For the TSPN solutions, TSPN-845 with a larger disk radius may lead to a shorter path by loosening the

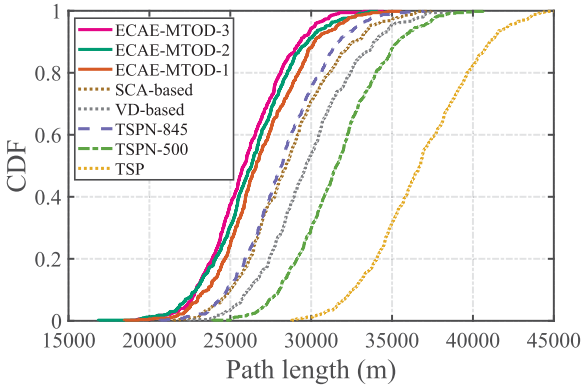


Fig. 11. CDF of UAV trajectory length when $\phi = 20$.

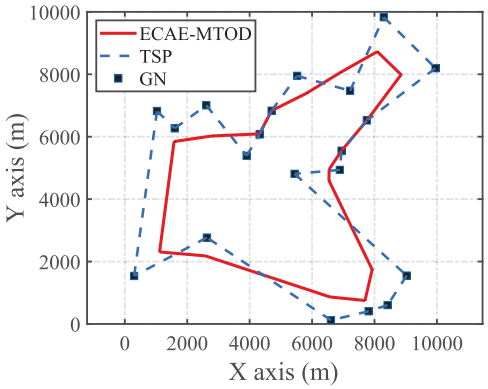


Fig. 12. An example of optimized paths in the ECAE-MTOD and TSP solutions.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.

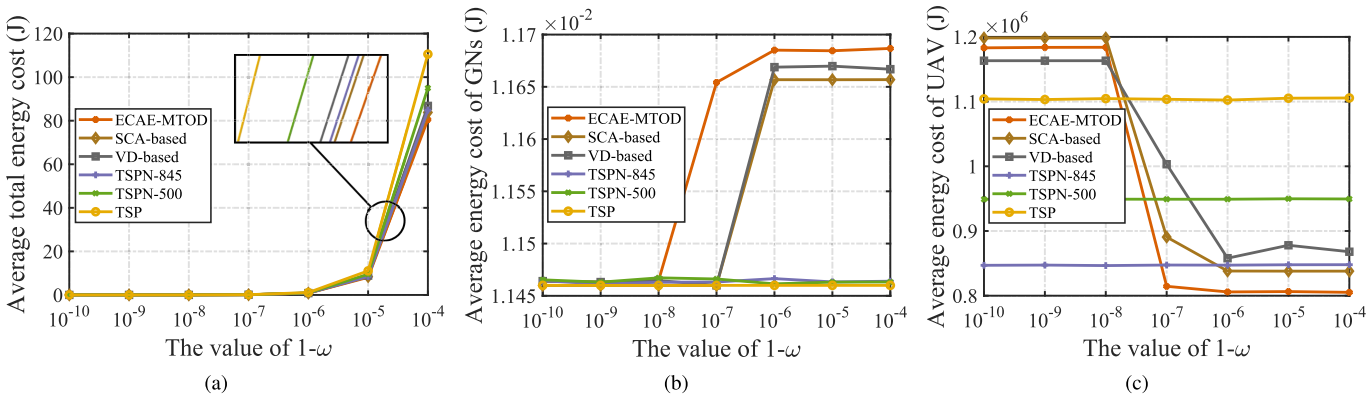


Fig. 13. (a), (b) and (c) respectively compare the means of E_{total}, E_{gn} and E_{uav} under different ω.

constraints on the range of DCLs, thus performing better than TSPN-500. We also observe that, the performance gap between the ECAE-MTOD scheme and TSPN-845 solution gradually narrows as the number of GNs increases. The reason is that when nodes are densely distributed, their coverage overlaps and the UAV would preferentially pass through overlapping regions to reduce the flight length. For the ECAE problem, actually the searching radius for DCLs does not reach D_{max}, and likely resembles that of TSPN-845, thus leading to similar results.

2) Impact of MHCR on UAV Trajectory Length: To ensure the quality of collected data, it is necessary to set a MHCR limit of GNs. From Fig. 5, if we adjust D_{max} according to the cases of PRR=90%, 50% and 30%, the corresponding values of D_{max} are 1122.2 m, 1273.5 m and 1324.6 m, and the corresponding probabilities of successful delivery within M times are 99.9%, 87.5%, and 65.7% respectively.

We denote the above three cases when ϕ=20 as ECAE-MTOD-1, ECAE-MTOD-2 and ECAE-MTOD-3 respectively. Fig. 11 compares the CDF of UAV path length of the proposed scheme with other algorithms. It is observed that ECAE-MTOD-1 can achieve a 27.1% reduction in average path length as compared with the TSP solution while guaranteeing the overall PRR of at least 99.9%. Specially, Fig. 12 shows an instance of optimized paths in the ECAE-MTOD and TSP solutions when $\phi = 20$ and $D_{max} = 1122.2$ m, where the path of ECAE-MTOD is visibly shorter and smoother than that of TSP. Moreover, in Fig. 11, ECAE-MTOD-1 can reduce the average path length by 6.6%, 10.2%, 4.9% and 15.1% as compared to the SCA-based algorithm, VD-based algorithm, TSPN-845 and TSPN-500 solutions respectively. The numerical results are similar to that of Fig. 10(a) mentioned above because UAV flight energy consumption accounts for a huge proportion in the total energy cost even though $\omega = 0.99$.

Compared to ECAE-MTOD-1, the average path length of ECAE-MTOD-3 and ECAE-MTOD-2 are reduced by 3.2% and 2.0%. Such gains are attributed to the expansion of GNs' coverage range (vary from 1122.2m to 1273.5m and 1324.6m), but the changes are too small as compared to the scale of collection area (10km), thus yielding a slight improvement. Note that both ECAE-MTOD-3 and ECAE-MTOD-2 reduce the path length by sacrificing some integrity and accuracy of data. Users can flexibly adjust D_{max} based on practical demands.

3) Impact of ω on Energy Consumption: In Eq. (12), ω reflects the importance we attach to E_{uav} or E_{gn}. Here, we fix $D_{max} = 1122.2$m, $\phi = 20$, and vary the value of ω from $1-10^{-4}$ to $1-10^{-10}$. The reason why ω is set so large is that E_{uav} and E_{gn} could differ by 10^7 or 10^8 orders of magnitude based on our configurations (see Fig. 13(b) and 13(c)).

Fig. 13(a) depicts the average E_{total} under different ω. As ω decreases, the proportion of E_{uav} increases, resulting in a distinct growth of E_{total}. The ECAE-MTOD scheme, as expected, always outperforms other solutions. For example, when $\omega = 1 - 10^{-4}$, the ECAE-MTOD scheme can reduce the average E_{total} by 27.2%, 15.2%, 5.0%, 4.1% and 7.3% as compared to the TSP, TSPN-500, TSPN-845, the SCA-based and VD-based algorithms. Meanwhile, the ECAE-MTOD scheme does not do much damage to the overall PRR owing to the reasonable coverage radius.

Fig. 13(b) shows the comparisons of average E_{gn} versus ω. The E_{gn} of TSP, TSPN-500 and TSPN-845 almost do not change since all the three algorithms can achieve a 100% PRR for each GN (i.e., GNs consume no energy for retransmissions, and E_{gn} cannot be further optimized). Therefore, no matter how ω changes, they merely focus on optimizing the path length. However, we trade off the benefit of reduced path length against the harm of reduced PRR by adjusting ω, thus, ω really matters for the ECAE problem. In Fig. 13(b), the E_{gn} of ECAE-MTOD is a little higher than others when $\omega < 1 - 10^{-7}$. As ω varies from $1 - 10^{-7}$ to $1 - 10^{-8}$, which corresponds to the magnitude difference between E_{gn} and E_{uav}, the E_{gn} of ECAE-MTOD sharply declines. The reason is that at this moment $\omega \cdot E_{gn}$ is in the same order of magnitude as $(1-\omega) \cdot E_{uav}$. Subsequently, the focus of optimization happens to shift.

Fig. 13(c) shows the comparisons of average E_{uav} versus ω. Similarly, the changes of ω do not have much impact on the E_{uav} of TSP, TSPN-500 and TSPN-845 since their E_{gn} always keep constant. Compared with the SCA-based algorithm, ECAE-MTOD can always achieve a lower E_{uav} since the SCA-based method neglects to optimize the visiting order of GNs. Although VD-based method can generate a feasible solution with low computational complexity, it may not effectively search the optimal one from the large solution

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.

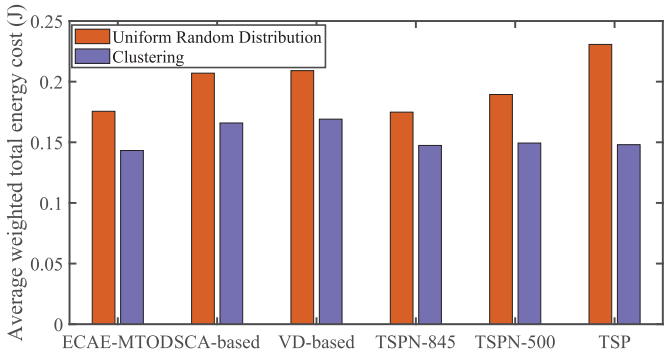


Fig. 14. The average E_{total} under two node distributions.

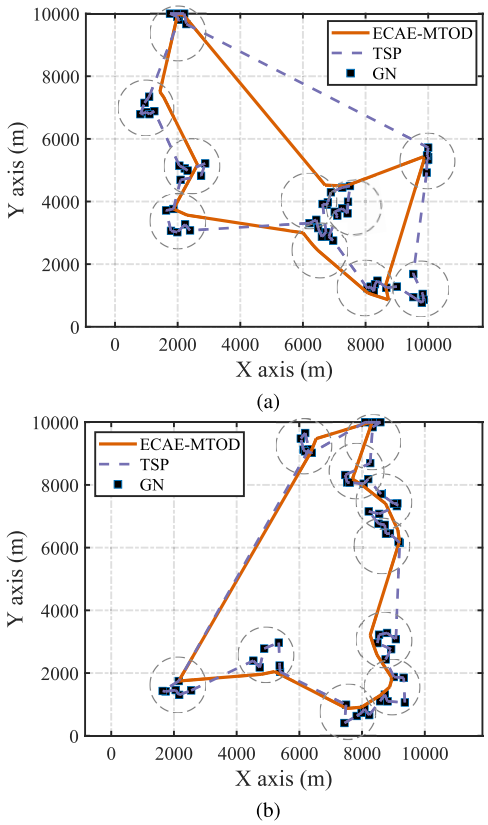


Fig. 15. Two clustering instances of optimized paths in the ECAE-MTOD and TSP solutions.

space. For the ECAE-MTOD scheme, as ω decreases, E_{uav} becomes more important in the objective function, and this part of energy consumption will be significantly optimized. In general, choosing ω between $1-10^{-7}$ and $1-10^{-8}$ can make both E_{gn} and E_{uav} at a low level.

4) Impact of Node Distribution on Total Energy Consumption: As aforementioned, we assume that the nodes are uniformly and randomly distributed in the experimental area. It is reasonable because adjacent farmlands usually have similar characteristics of temperature, humidity, etc, which means the data collected from the clustered sensors is redundant. However, the farmlands are usually composed of spatially dispersed fragments in practice. Therefore, we consider the case of node clustering distribution, where GNs are distributed in many isolated pieces of farmland.

We assume that each cluster's nodes are sampled from a Gaussian distribution *normal(v, std)*, where v is the mean and std is the standard deviation (we fix $std = 0.1$). Each Gaussian distribution's v is randomly generated, which determines the position of each cluster. Here, we consider the case of $\phi = 60$, the number of clusters $N_c = 10$ and there are 6 GNs in each cluster. And we set $\omega = 1-10^{-7}$ according to the analysis of Fig. 13.

In Fig. 14, we see that the average E_{total} of uniform distribution for the six methods is generally higher than that of clustering. The ECAE-MTOD scheme can reduce the E_{total} of clustering by 18.5% as compared to that of uniform distribution. Moreover, the ECAE-MTOD scheme achieves the minimum E_{total} under two node distributions. For clustering, the reduction percentages are 15.3%, 13.7%, 2.8%, 4.1% and 3.2% as compared to the VD- based algorithm, SCA-based algorithm, TSPN-845, TSPN-500 and TSP respectively. Fig. 15 (a) and (b) show two clustering instances of optimized paths in the ECAE-MTOD and TSP solutions. Comparing Fig. 11 and Fig. 15, the optimization performance of ECAE-MTOD for the uniform distribution is more remarkable. The reason is that the clustered nodes are relatively concentrated distributed, which benefits the path optimization.

IX. Conclusion

In this article, we investigate the joint optimization problem of UAV trajectory length and data collection efficiency. Our objective is to find a flight trajectory constituted by a set of data collection locations to maximize the overall PRR under the energy constraints of both UAV and sensors. Taking the physical layer mechanism of LoRa into consideration, we propose a PRR-based coverage model to measure the transmission performance. As the formulated ECAE problem is NP-hard and difficult to solve in a timely manner, we present a DRL-based scheme to make fast decisions on the ECAE problem for different node deployments. In particular, an attention-based model is trained to quickly output an initial solution. An intuitive optimization algorithm is then devised to further optimize the model outputs. Extensive simulations show the proposed scheme can efficiently reduce the total energy cost by 27.1% as compared with the best baseline's policy while keeping a satisfactory PRR.

Appendix A
Attention Mechanism

The Transformer architecture [40] is one of the most impactful members of the attention family. The transformer interprets the encoded representation of inputs as a set of *key-value* pairs $(\boldsymbol{K}, \boldsymbol{V})$ with dimensions d_k and d_v, and maps a *query* $\boldsymbol{Q}$ and these key-value pairs to an output. For each node, the query $\boldsymbol{q_i}$, key $\boldsymbol{k_i}$, and value $\boldsymbol{v_i}$ vectors are obtained by linearly transforming node embedding $\boldsymbol{h_i}$:

$$\boldsymbol{q_i} = \boldsymbol{W^Q h_i}, \boldsymbol{k_i} = \boldsymbol{W^K h_i}, \boldsymbol{v_i} = \boldsymbol{W^V h_i}, \tag{18}$$

where $\boldsymbol{W^Q}$, $\boldsymbol{W^K}$ and $\boldsymbol{W^V}$ are parameter matrices to be learned. The transformer adopts the scaled dot-product attention, adding a scaling factor $\frac{1}{\sqrt{d_k}}$ as compared to dot-product:

$$Attention(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = softmax(\frac{\boldsymbol{Q K^T}}{\sqrt{d_k}})\boldsymbol{V}, \tag{19}$$

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.

where $\boldsymbol{Q} = [\boldsymbol{q_1}, \ldots, \boldsymbol{q_K}]$, $\boldsymbol{K} = [\boldsymbol{k_1}, \ldots, \boldsymbol{k_K}]$, $\boldsymbol{V} = [\boldsymbol{v_1}, \ldots, \boldsymbol{v_K}]$. Note that if node i is not adjacent to node j, $\frac{\boldsymbol{q_i} \cdot \boldsymbol{k_j}^T}{\sqrt{d_k}}$ is replaced by $-\infty$ to impede messages passing between two nodes.

APPENDIX B
MULTI-HEAD ATTENTION

Instead of computing the attention only once, the multi-head attention runs through the scaled dot-product attention several times in parallel. Finally, N_M (the number of heads) independent attention outputs are simply concatenated and linearly transformed into the expected dimensions:

$$\begin{aligned} &MultiHead(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = [\boldsymbol{head_1}, \ldots, \boldsymbol{head_{N_m}}]\boldsymbol{W^O}, \\ &\boldsymbol{head_i} = Attention(\boldsymbol{QW_i^Q}, \boldsymbol{KW_i^K}, \boldsymbol{VW_i^V}), \end{aligned} \tag{20}$$

where the above $\boldsymbol{W}$s are all learnable parameter matrices.

APPENDIX C
FEED-FORWARD SUBLAYER

The feed-forward sublayer aims to process the concatenated output from different attention heads. The feed-forward sublayer includes a hidden layer and a ReLu activation:

$$FF(\boldsymbol{h_i^\wedge}) = \boldsymbol{W^{ff,1}} \cdot \mathrm{ReLU}(\boldsymbol{W^{ff,0}} \cdot \boldsymbol{h_i^\wedge} + \boldsymbol{b^{ff,0}}) + \boldsymbol{b^{ff,1}}. \tag{21}$$

REFERENCES

[1] A. Caruso, S. Chessa, S. Escolar, J. Barba, and J. C. López, "Collection of data with drones in precision agriculture: Analytical model and LoRa case study," *IEEE Internet Things J.*, vol. 8, no. 22, pp. 16692–16704, Nov. 2021.

[2] B. Kong, H. Huang, and X. Jia, "Path planning for sensor data collection by using UAVs," in *Proc. IEEE MSN*, Dec. 2018, pp. 199–205.

[3] C. Luo, M. N. Satpute, D. Li, Y. Wang, W. Chen, and W. Wu, "Fine-grained trajectory optimization of multiple UAVs for efficient data gathering from WSNs," *IEEE/ACM Trans. Netw.*, vol. 29, no. 1, pp. 162–175, Feb. 2021.

[4] M. B. Ghorbel, D. Rodríguez-Duarte, H. Ghazzai, Md. J. Hossain, and H. Menouar, "Joint position and travel path optimization for energy efficient wireless data gathering using unmanned aerial vehicles," *IEEE Trans. Veh. Technol.*, vol. 68, no. 3, pp. 2165–2175, Mar. 2019.

[5] B. Wang, "Coverage problems in sensor networks: A survey," *ACM Comput. Surv.*, vol. 43, no. 4, pp. 1–53, Oct. 2011.

[6] Y. He et al., "Deep-reinforcement-learning-based optimization for cache-enabled opportunistic interference alignment wireless networks," *IEEE Trans. Veh. Technol.*, vol. 66, no. 11, pp. 10433–10445, Nov. 2017.

[7] C. H. Papadimitriou, "The Euclidean travelling salesman problem is NP-complete," *Theor. Comput. Sci.*, vol. 4, no. 3, pp. 237–244, Jun. 1977.

[8] J. Gudmundsson and C. Levcopoulos, "Hardness result for TSP with neighborhoods," Dept. Comput. Sci., Lund Univ., Lund, Sweden, Tech. Rep. LU-CS-TR: 2000-216, 2000.

[9] R. Ding, F. Gao, and X. S. Shen, "3D UAV trajectory design and frequency band allocation for energy-efficient and fair communication: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 19, no. 12, pp. 7796–7809, Dec. 2020.

[10] N. Zhao, Y. Cheng, Y. Pei, Y.-C. Liang, and D. Niyato, "Deep reinforcement learning for trajectory design and power allocation in UAV networks," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Jun. 2020, pp. 1–6.

[11] W. Zhang, Q. Wang, X. Liu, Y. Liu, and Y. Chen, "Three-dimension trajectory design for multi-UAV wireless network with deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 70, no. 1, pp. 600–612, Jan. 2021.

[12] M. T. Nguyen and K. A. Teague, "Tree-based energy-efficient data gathering in wireless sensor networks deploying compressive sensing," in *Proc. 23rd Wireless Opt. Commun. Conf. (WOCC)*, May 2014, pp. 1–6.

[13] R. Huang, L. Ma, G. Zhai, J. He, X. Chu, and H. Yan, "Resilient routing mechanism for wireless sensor networks with deep learning link reliability prediction," *IEEE Access*, vol. 8, pp. 64857–64872, 2020.

[14] S. Goudarzi, N. Kama, M. H. Anisi, S. Zeadally, and S. Mumtaz, "Data collection using unmanned aerial vehicles for Internet of Things platforms," *Comput. Electr. Eng.*, vol. 75, pp. 1–15, May 2019.

[15] Z. Hao, N. Qu, X. Dang, and J. Hou, "RSS-based coverage deployment method under probability model in 3D-WSN," *IEEE Access*, vol. 7, pp. 183091–183104, 2019.

[16] S. Kumar and D. K. Lobiyal, "Sensing coverage prediction for wireless sensor networks in shadowed and multipath environment," *Sci. World J.*, vol. 2013, Oct. 2013, Art. no. 565419.

[17] O. Gnawali, R. Fonseca, K. Jamieson, D. Moss, and P. Levis, "Collection tree protocol," in *Proc. 7th ACM Conf. Embedded Networked Sensor Syst.*, Nov. 2009, pp. 1–14.

[18] M. Samir, S. Sharafeddine, C. M. Assi, T. M. Nguyen, and A. Ghrayeb, "UAV trajectory planning for data collection from time-constrained IoT devices," *IEEE Trans. Wireless Commun.*, vol. 19, no. 1, pp. 34–46, Jan. 2020.

[19] Y. Zeng, J. Xu, and R. Zhang, "Energy minimization for wireless communication with rotary-wing UAV," *IEEE Trans. Wireless Commun.*, vol. 18, no. 4, pp. 2329–2345, Apr. 2019.

[20] N. Zhao et al., "Joint trajectory and precoding optimization for UAV-assisted NOMA networks," *IEEE Trans. Commun.*, vol. 67, no. 5, pp. 3723–3735, May 2019.

[21] J. Baek, S. I. Han, and Y. Han, "Energy-efficient UAV routing for wireless sensor networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 1741–1750, Feb. 2020.

[22] H. Bayerlein, M. Theile, M. Caccamo, and D. Gesbert, "UAV path planning for wireless data harvesting: A deep reinforcement learning approach," in *Proc. IEEE GLOBECOM*, Jul. 2020, pp. 1–6.

[23] K. Li, W. Ni, E. Tovar, and A. Jamalipour, "On-board deep Q-network for UAV-assisted online power transfer and data collection," *IEEE Trans. Veh. Technol.*, vol. 68, no. 12, pp. 12215–12226, Dec. 2019.

[24] A. Ferdowsi, M. A. Abd-Elmagid, W. Saad, and H. S. Dhillon, "Neural combinatorial deep reinforcement learning for age-optimal joint trajectory and scheduling design in UAV-assisted networks," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 5, pp. 1250–1265, May 2021.

[25] M. A. Abd-Elmagid and H. S. Dhillon, "Average peak age-of-information minimization in UAV-assisted IoT networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 2, pp. 2003–2008, Feb. 2019.

[26] C. H. Liu, Z. Chen, J. Tang, J. Xu, and C. Piao, "Energy-efficient UAV control for effective and fair communication coverage: A deep reinforcement learning approach," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 9, pp. 2059–2070, Sep. 2018.

[27] K. Srinivasan, M. A. Kazandjieva, S. Agarwal, and P. Levis, "The β-factor: Measuring wireless link burstiness," in *Proc. 6th ACM Conf. Embedded Netw. Sensor Syst.*, Nov. 2008, pp. 29–42.

[28] J. C. Liando, A. Gamage, A. W. Tengourtius, and M. Li, "Known and unknown facts of LoRa: Experiences from a large-scale measurement study," *ACM Trans. Sensor Netw.*, vol. 15, no. 2, pp. 1–35, May 2019.

[29] T. S. Rappaport, *Wireless Communications—Principles and Practice*. Upper Saddle River, NJ, USA: Prentice-Hall, 1996.

[30] T. Elshabrawy and J. Robert, "Closed-form approximation of LoRa modulation BER performance," *IEEE Commun. Lett.*, vol. 22, no. 9, pp. 1778–1781, Sep. 2018.

[31] G. Ferré and A. Giremus, "LoRa physical layer principle and performance analysis," in *Proc. 25th IEEE Int. Conf. Electron., Circuits Syst. (ICECS)*, Dec. 2018, pp. 65–68.

[32] J. Tapparel, O. Afisiadis, P. Mayoraz, A. Balatsoukas-Stimming, and A. Burg, "An open-source LoRa physical layer prototype on GNU radio," in *Proc. IEEE 21st Int. Workshop Signal Process. Adv. Wireless Commun. (SPAWC)*, May 2020, pp. 1–5.

[33] H. Mroue, A. Nasser, B. Parrein, S. Hamrioui, E. Mona-Cruz, and G. Rouyer, "Analytical and simulation study for LoRa modulation," in *Proc. 25th Int. Conf. Telecommun. (ICT)*, Jun. 2018, pp. 655–659.

[34] D. H. Kim, J. Y. Lim, and J. D. Kim, "Low-power, long-range, high-data transmission using Wi-Fi and LoRa," in *Proc. 6th Int. Conf. IT Converg. Secur. (ICITCS)*, Sep. 2016, pp. 1–3.

[35] A. Gamage, J. C. Liando, C. Gu, R. Tan, and M. Li, "LMAC: Efficient carrier-sense multiple access for LoRa," in *Proc. 26th Annu. Int. Conf. Mobile Comput. Netw.*, Sep. 2020, pp. 1–13.

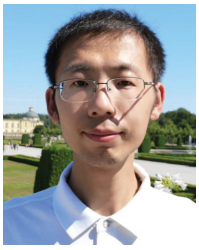
[36] K. Xu et al., "Show, attend and tell: Neural image caption generation with visual attention," in *Proc. 32nd Int. Conf. Mach. Learn.*, Mar. 2015, pp. 2048–2057.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.

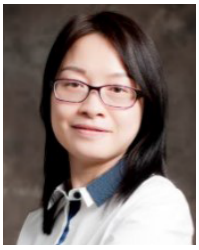
[37] W. Kool, H. Van Hoof, and M. Welling, "Attention, learn to solve routing problems!" in *Proc. ICLR*, New Orleans, LA, USA, May 2019, pp. 1–25.
[38] M. Nazari, A. Oroojlooy, L. V. Snyder, and M. Takáč, "Reinforcement learning for solving the vehicle routing problem," in *Proc. Adv. Neural Inf. Process. Syst.*, Dec. 2018, pp. 9861–9871.
[39] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, Feb. 2018, pp. 1–12.
[40] A. Vaswani et al., "Attention is all you need," in *Proc. NIPS*, Jun. 2017, pp. 5998–6008.
[41] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
[42] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn.*, Jul. 2015, pp. 448–456.
[43] I. Bello, H. Pham, Q. V. Le, M. Norouzi, and S. Bengio, "Neural combinatorial optimization with reinforcement learning," in *Proc. Int. Conf. Learn. Represent.*, Apr. 2017, pp. 1–15.
[44] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 229–256, May 1992.
[45] D. P. Kingma and J. L. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent.*, May 2015, pp. 1–15.
[46] D. Applegate, R. Bixby, V. Chvatal, and W. Cook. *Concorde TSP Solver*. Accessed: Oct. 7, 2021. [Online]. Available: https://en-academic.com/dic.nsf/enwiki/9431653
[47] M. Deudon, P. Cournut, A. Lacoste, Y. Adulyasak, and L.-M. Rousseau, "Learning heuristics for the TSP by policy gradient," in *Proc. Int. Conf. Integr. Constraint Program., Artif. Intell., Oper. Res. (CPAIOR)*, Jun. 2018, pp. 170–181.
[48] F. Shan, J. Luo, R. Xiong, W. Wu, and J. Li, "Looking before crossing: An optimal algorithm to minimize UAV energy by speed scheduling with a practical flight energy model," in *Proc. IEEE INFOCOM*, Jul. 2020, pp. 1758–1767.
[49] I. Gentilini, F. Margot, and K. Shimada, "The travelling salesman problem with neighbourhoods: MINLP solution," *Optim. Methods Softw.*, vol. 28, no. 2, pp. 364–378, Apr. 2013.



Yuchen Zhu received the B.S. degree in software engineering from Wuhan University, Hubei, China, in 2018. She is currently pursuing the Ph.D. degree with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China. Her current research interests include neighbor discovery algorithms and unmanned aerial vehicle networks.



Bo Yang (Member, IEEE) received the Ph.D. degree in control science and engineering from the University of Chinese Academy of Sciences, Beijing, China, in 2017. He is currently a Full Professor with the College of Information Engineering, Northwest A&F University, China. His current research interests include wireless networks, multi-agent learning, and coordination.



Min Liu (Senior Member, IEEE) received the B.S. and M.S. degrees in computer science from Xi'an Jiaotong University in 1999 and 2002, respectively, and the Ph.D. degree in computer science from the Graduate University of Chinese Academy of Sciences in 2008. She is currently a Professor with the Institute of Computing Technology, Chinese Academy of Sciences, and also holds a position with the Zhongguancun Laboratory. Her current research interests include mobile computing and edge intelligence.



Zhongcheng Li received the B.S. degree in computer science from Peking University, Beijing, China, in 1983, and the M.S. and Ph.D. degrees from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, in 1986 and 1991, respectively. From 1996 to 1997, he was a Visiting Professor with the University of California at Berkeley, CA, USA. He is currently a Professor with the Institute of Computing Technology, Chinese Academy of Sciences. His current research interests include computer networks and communications.

Authorized licensed use limited to: Nanjing Univ of Post & Telecommunications. Downloaded on October 10,2026 at 07:03:55 UTC from IEEE Xplore. Restrictions apply.